POSTGRESQL У PRODUCTION

Від одного instance до масштабованої, відмовостійкої та відновлюваної production-системи

Заняття №15

Node.js Pro

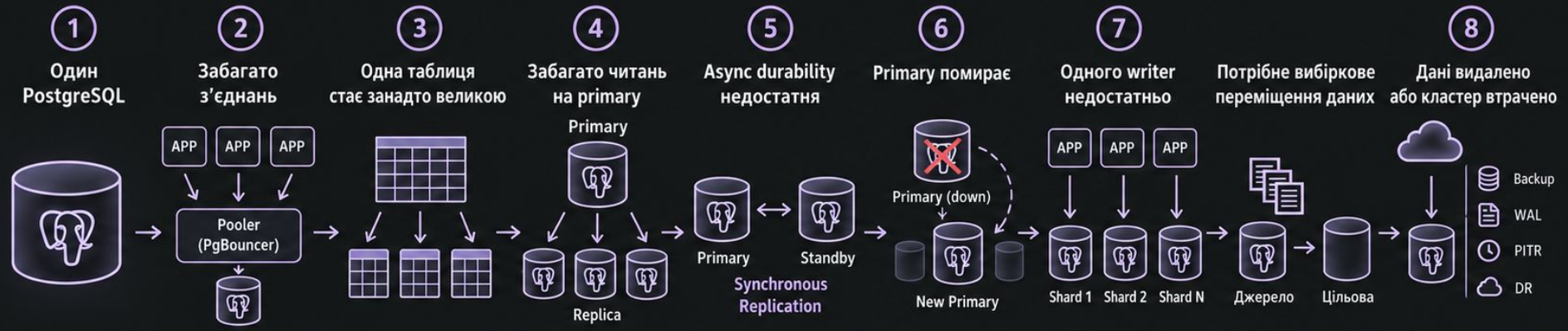
Лектор
Кирило Орлов

r_d

Еволюція PostgreSQL до production-рівня

ВІД ОДНІЄЇ БАЗИ ДАНИХ ДО НАДІЙНОЇ, МАСШТАБОВАНОЇ PRODUCTION-ПЛАТФОРМИ

Та сама база. Більші проблеми. Сильніші рішення.



Одна база даних обслуговує ваш додаток.

Просто працює на початку.

Прямі з'єднання не масштабуються. Ви досягаєте лімітів з'єднань.

Pool / PgBouncer

Використовуйте пул з'єднань (наприклад, PgBouncer).

Одна таблиця виростає надто великою, що спричиняє повільні запити та проблеми з обслуговуванням.

Partitioning

Розбийте таблицю на партиції (наприклад, за часом або ключем).

Читання перевантажує primary, збільшуючи затримки.

Replication / Read Replicas

Розвантажте читання на replica.

Асинхронна реплікація може призвести до втрати даних під час збою (навіть рідко).

Synchronous Replication

Оцініть підтвердження запису на standby.

Primary може вийти з ладу через збій обладнання, мережі або людську помилку.

Failover / HA

Автоматично перемикайтеся на доступний вузол (наприклад, Patroni, Kubernetes, cloud).

Один primary не може обробити безперервне зростання запитів або обсягу даних.

Sharding

Розподіліть дані між кількома primary (шарадами).

Потрібно переміщати вибіркові дані (наприклад, схеми тенанти, таблиці або рядки на нові сервіси) без простою.

Logical Replication

Реплікуйте вибрані таблиці або рядки між кластерами.

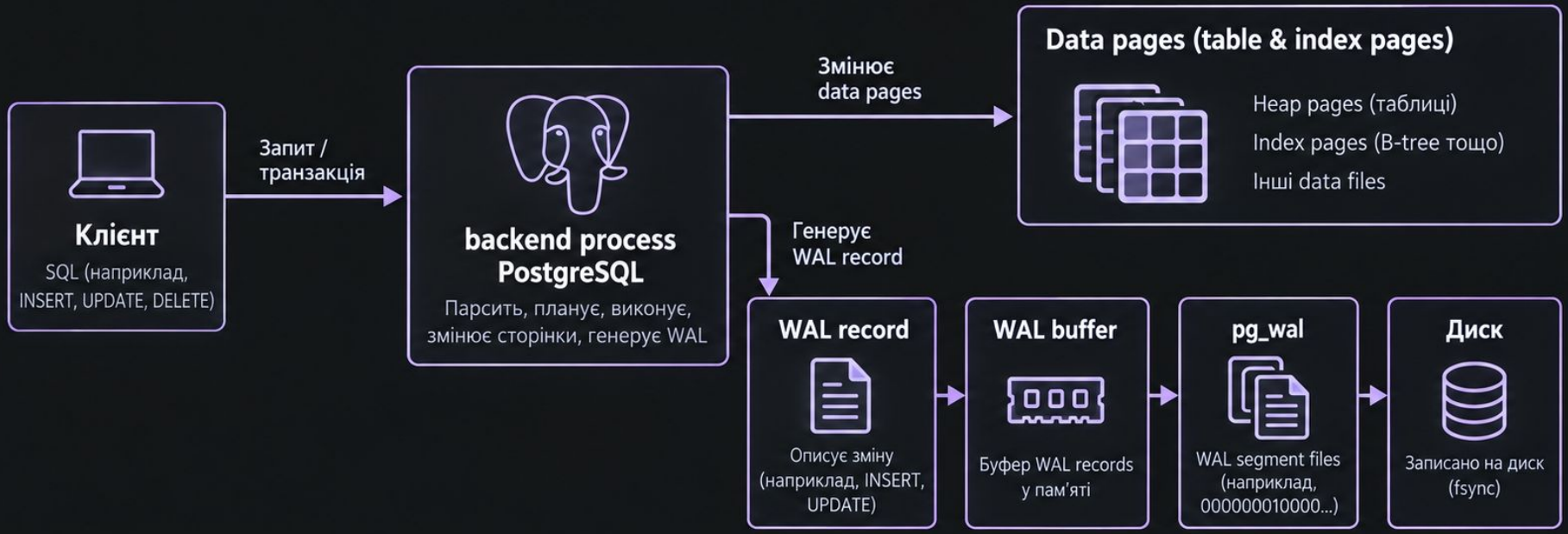
Дані можуть бути видалені (випадково або зловмисно) або весь кластер може бути втрачено.

Backup + WAL + PITR + DR

Регулярні резервні копії, архівація WAL, відновлення на певний момент часу (PITR) і план відновлення після катастроф (DR).

PostgreSQL + WAL

Спочатку записуємо зміни у WAL, а потім застосовуємо їх до data files.



Ключова ідея: WAL — це основа для crash recovery, replication і point-in-time recovery (PITR).

Усі зміни фіксуються в WAL до запису в data files, тому PostgreSQL може відновити базу після збою, стрімити зміни на replicas і відновитися до конкретного моменту часу.

- ✓ Crash recovery
- ✓ Replication
- ✓ Point-in-time recovery (PITR)

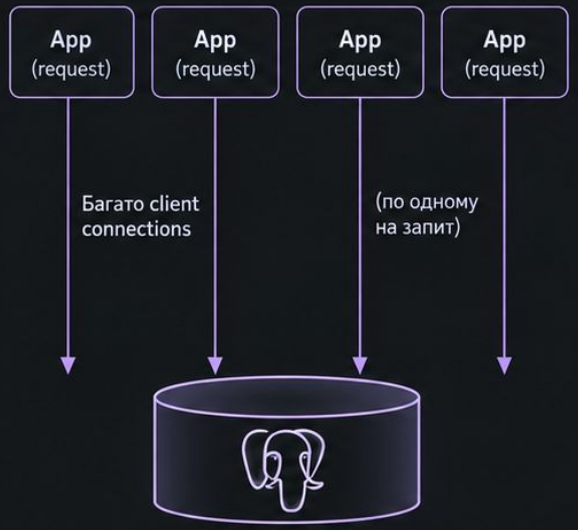
Масштабування з’єднань

Клієнтські з’єднання ≠ server connections PostgreSQL

Багато application requests і client connections можна мультиплексувати на невелику кількість server connections PostgreSQL.

1 ПОГАНО

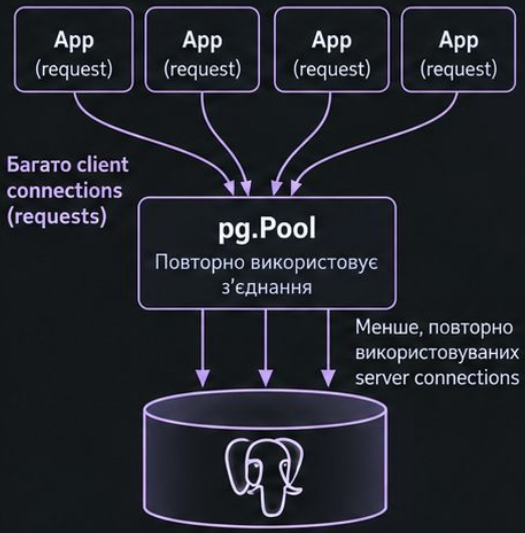
Кожен запит відкриває окреме з’єднання з PostgreSQL



Багато server connections
(високий overhead, не масштабується)

2 КРАЩЕ

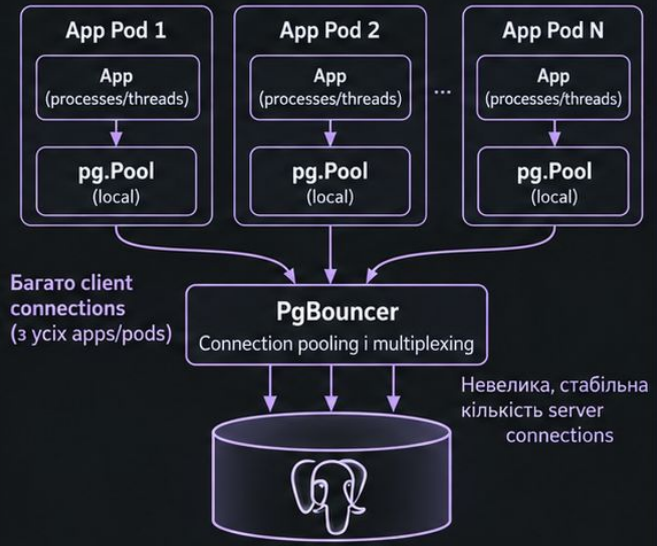
Використовуй connection pool в application



Менше server connections
(reuse)

3 PRODUCTION-ПАТЕРН

Кілька app processes/pods з локальними pools + PgBouncer



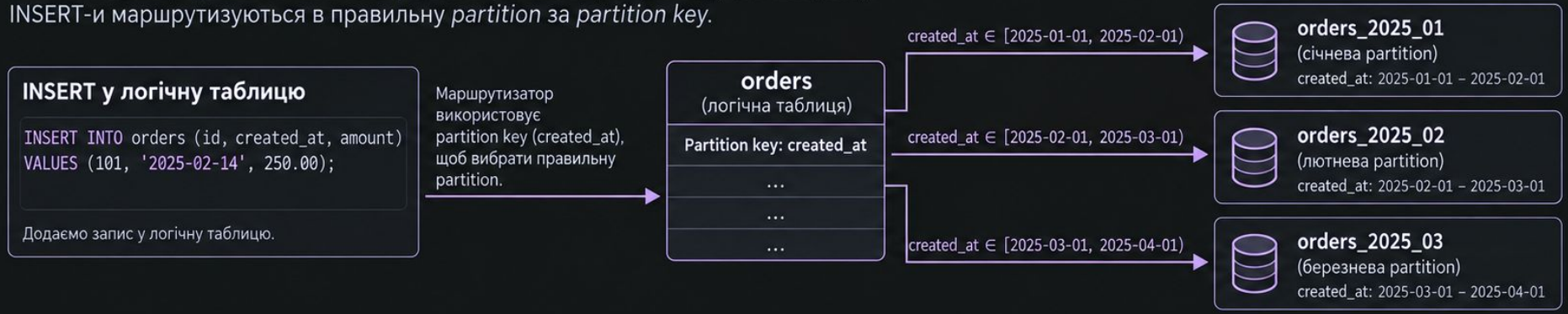
Мало server connections
(стабільно, добре масштабується)

Ключовий висновок: client connections ≠ server connections PostgreSQL.

Використовуй pooling (pg.Pool) і proxy на кшталт PgBouncer, щоб мультиплексувати багато client connections у невелику, стабільну кількість server connections PostgreSQL.

Партиціювання

Логічна таблиця ділиться на кілька фізичних partitions (наприклад, за місяцями).
INSERT-и маршрутизуються в правильну *partition* за *partition key*.



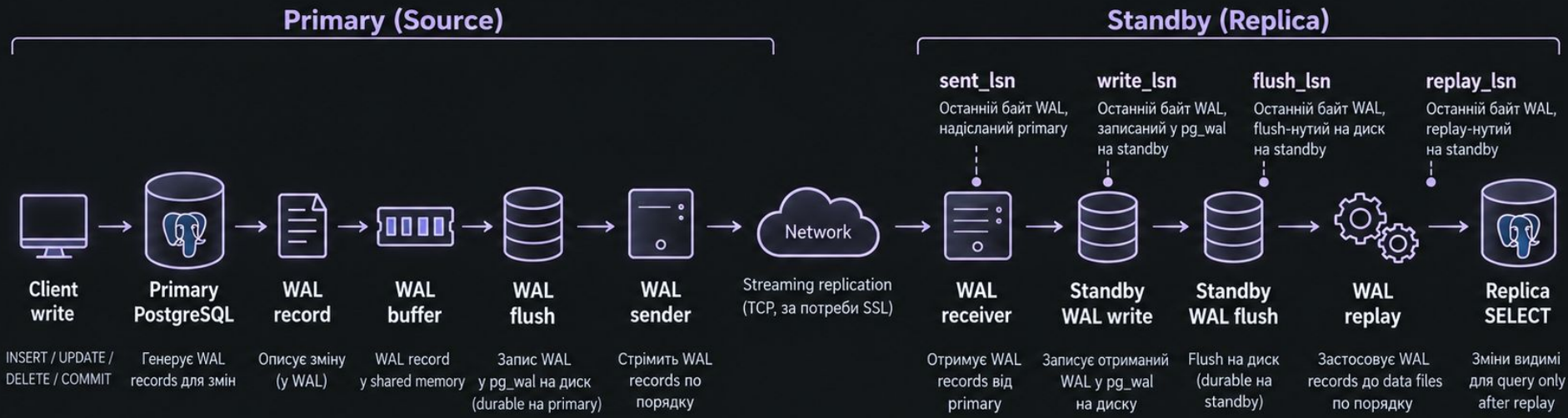
Query → partition pruning → Index Scan

Query з фільтром за created_at читає лише відповідну partition, а далі використовує index усередині неї, щоб знайти rows.



Конвеєр фізичної реплікації

WAL генерується на primary, стріміться на standby, записується і replay-иться.
Дані стають видимими на replica лише після replay, а не просто після отримання.



Властивості

- WAL записується і flush-нується на primary для надійності.
- WAL стріміться по порядку на standby.
- Standby відстежує прогрес через LSN: sent_lsn, write_lsn, flush_lsn, replay_lsn.
- Запити на replica бачать зміни тільки після replay.

Ключова ідея

Зміна стає видимою на replica тільки після того, як її replay-нуто (replay_lsn), а не тоді, коли її просто отримано чи записано.

Видимість на replica настає після replay

Зміни стають видимими при досягненні replay_lsn, а не просто після receive (write_lsn) чи flush (flush_lsn).

ЧОТИРИ СТАДІЇ LSN

sent_lsn · write_lsn · flush_lsn · replay_lsn

sent_lsn 1

PRIMARY ВІДПРАВИВ

Останній байт WAL, надісланий зі source.

write_lsn 2

REPLICA ЗАПИСАЛА

WAL уже у pg_wal / OS buffers standby.

flush_lsn 3

REPLICA НА ДИСКУ

WAL durable на диску standby.

replay_lsn 4

REPLICA ЗАСТОСУВАЛА

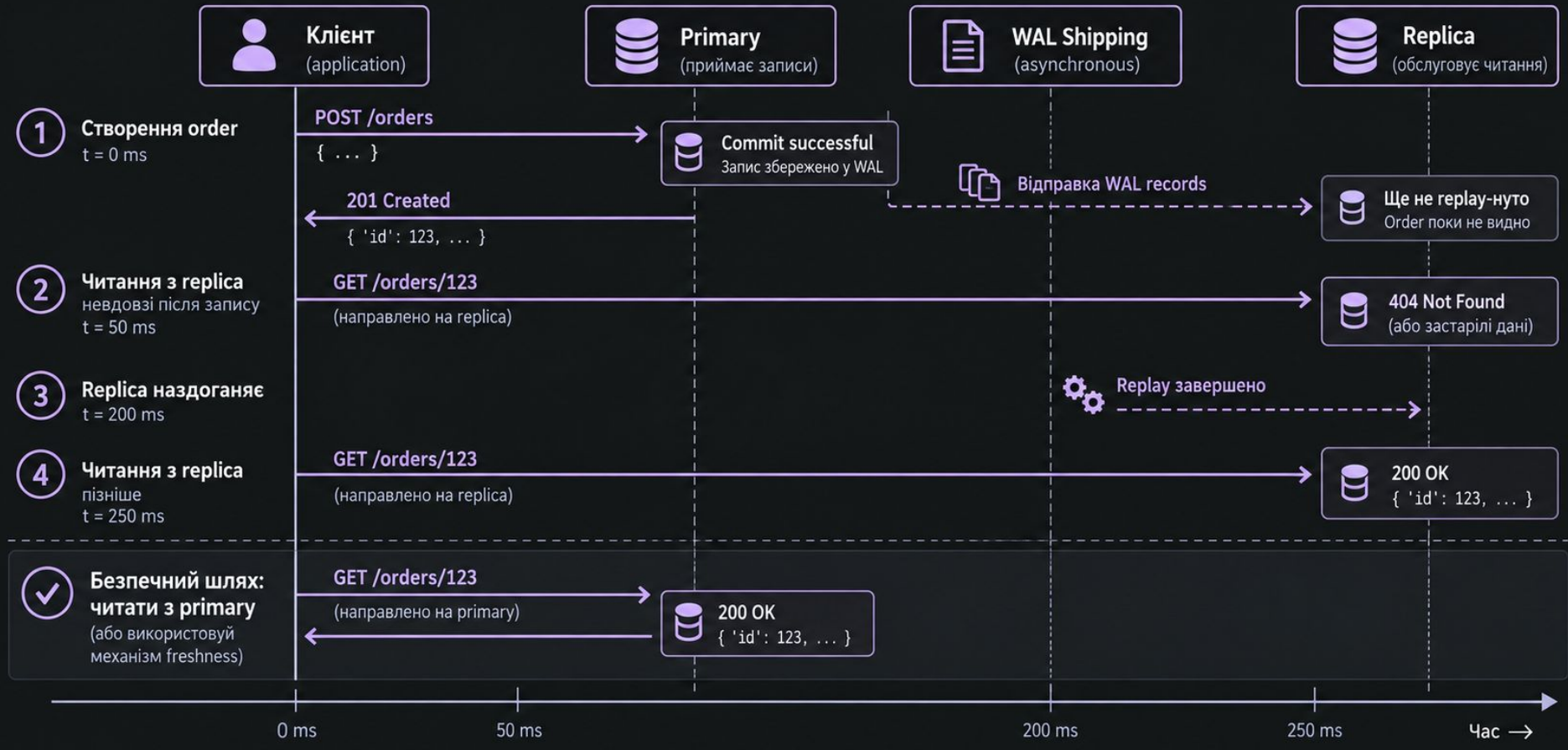
Тільки тепер зміна видима в SELECT.

sent ≠ written ≠ flushed ≠ replayed

Lag у байтах = різниця позицій у потоці WAL

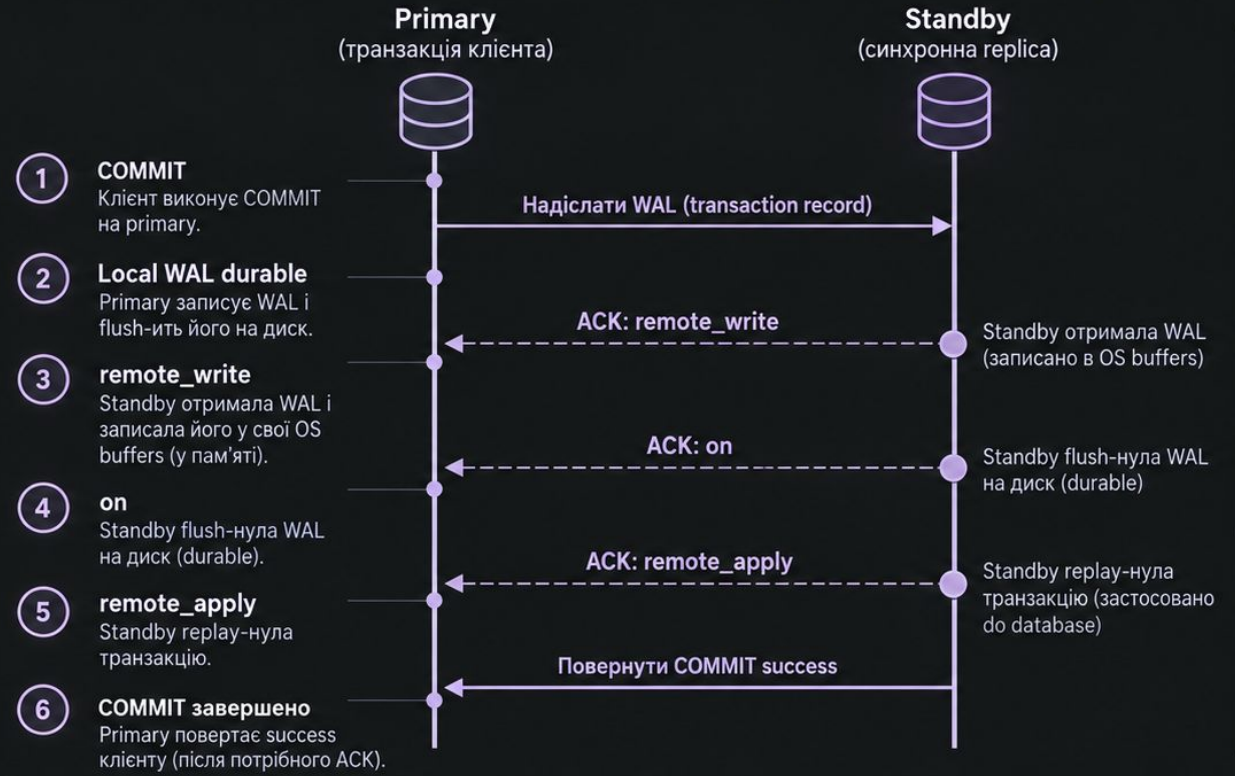
Проблема read-after-write

Запис, закомічений на primary, може бути не одразу видимим на replica.
Наступне читання з replica може повернути 404 або застарілі дані, доки replay не завершиться.



Синхронна реплікація

Primary чекає на підтвердження від standby, перш ніж COMMIT завершиться.



Рівні synchronous commit (на standby)

remote_write
Standby отримала WAL і записала його у свої OS buffers (у пам'яті).

Найнижча latency
Слабша гарантія
Вища availability

on
Standby flush-нула WAL на диск (durable).

Вища latency
Сильніша гарантія
Нижча availability

remote_apply
Standby replay-нула транзакцію (застосовано до database).

Найвища latency
Найсильніша гарантія
Найнижча availability

Сильніша гарантія durability ↔ вища latency / нижча availability

Чим більше ви чекаєте на standby, тим сильніша гарантія durability, але тим вища latency коміту і тим більший ризик недоступності, якщо standby працює повільно або недоступна.

ПЕРЕРВА

5 ХВИЛИН

Далі: availability → distributed writes → recovery

r_d

Failover і split brain

Failover має бути швидким. Але потрібно гарантувати, що old primary більше не може приймати записи.

1 Таймлайн failover (нормальна робота)

t0 — Primary healthy
Replica стримиться

t1 — Primary помирає
Primary недоступна

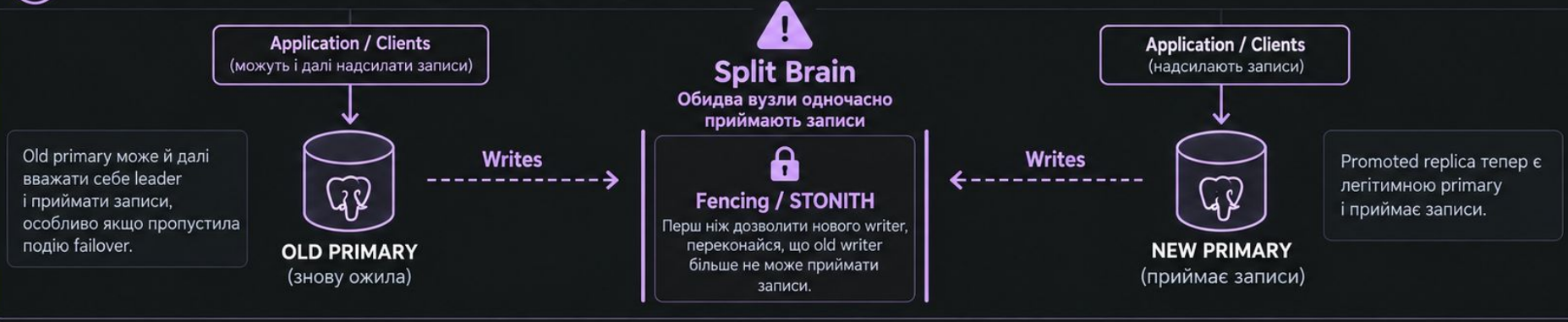
t2 — Збій виявлено
Health checks / quorum підтверджують, що primary недоступна

t3 — Promote replica
Replica промотується в new primary

t4 — Маршрутизація трафіку application
Applications надсилають записи в new primary

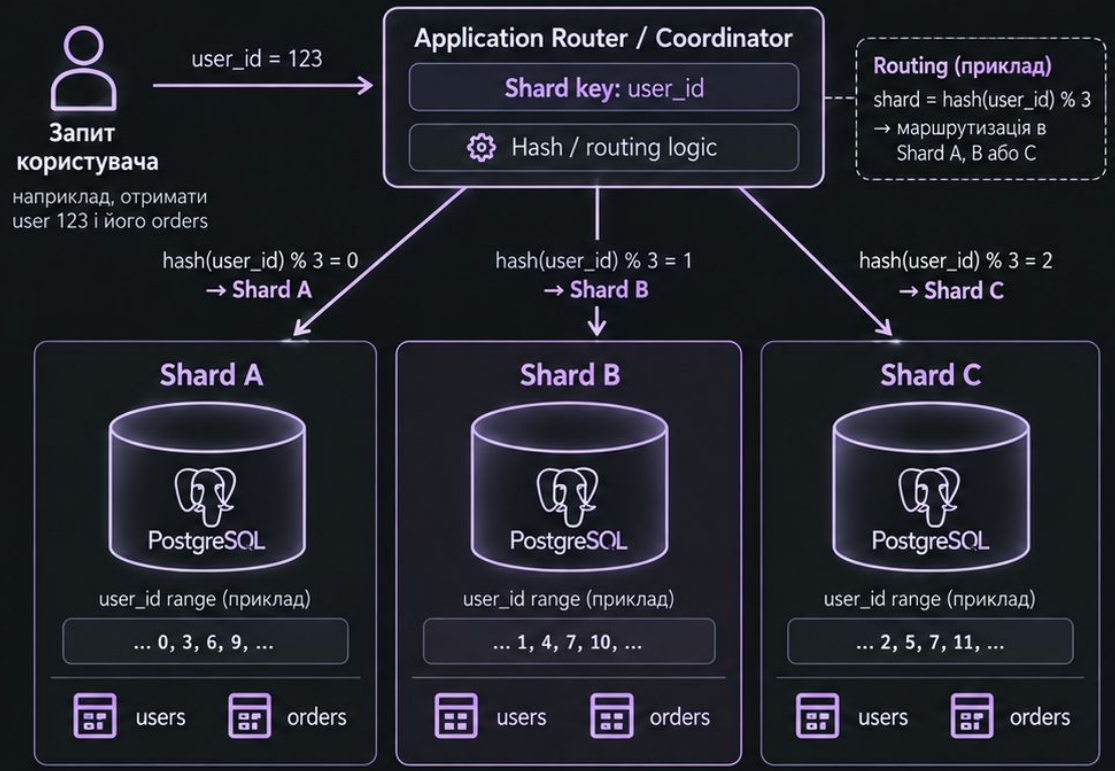


2 Небезпечний момент: old primary повертається



Шардінг

Розподіли свої дані (і workload) між кількома незалежними databases.
Шардінг ділить усю систему — а не лише одну таблицю.



Локальна робота (той самий shard) — легко

Користувач і його orders живуть в одному shard.
Простий локальний query/transaction — майже як у звичайній database.

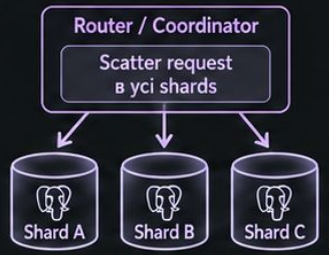



```
SELECT u.*, o.*
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE u.id = 123;
```

Одна database, local join/transaction.
Швидко і просто.

Cross-shard робота — дорого

Дані для багатьох users (або join між різними users) живуть у кількох shards. Потрібні scatter, network і merge.



1. **Scatter:** надіслати query в A, B, C
2. **Network:** кожен shard обробляє свою частину
3. **Merge:** coordinator об'єднує і сортує результати

Розподілений query, вища latency, більше складності (error handling, partial failures, consistency тощо).

Шардінг ділить усю систему між кількома databases — кожен shard є самодостатньою database (схема, дані, індекси, transactions), а application маршрутизує запити за shard key (наприклад, user_id).

RESHARDING: ЧОМУ % N БОЛИТЬ

modulo 4 → 5 проти consistent hashing

MODULO

17 % 4 = 1
17 % 5 = 2

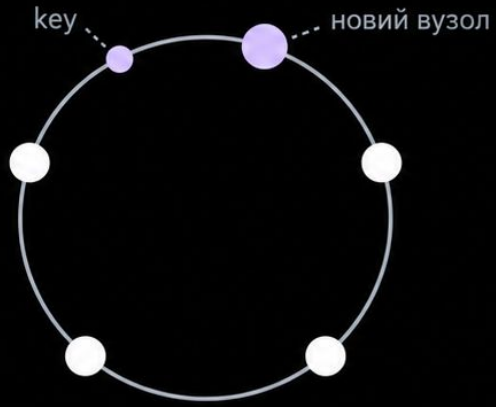
≈80% КЛЮЧІВ

змінюють bucket при 4 → 5



4/20 лишаються на тому самому shard

CONSISTENT HASHING



≈20% КЛЮЧІВ

переїжджають до нового вузла при 4 → 5

АЛЬТЕРНАТИВА: CONSISTENT HASHING АБО DIRECTORY MAPPING • tenant_id → shard_id

LOGICAL REPLICATION

обрані таблиці й row-level changes

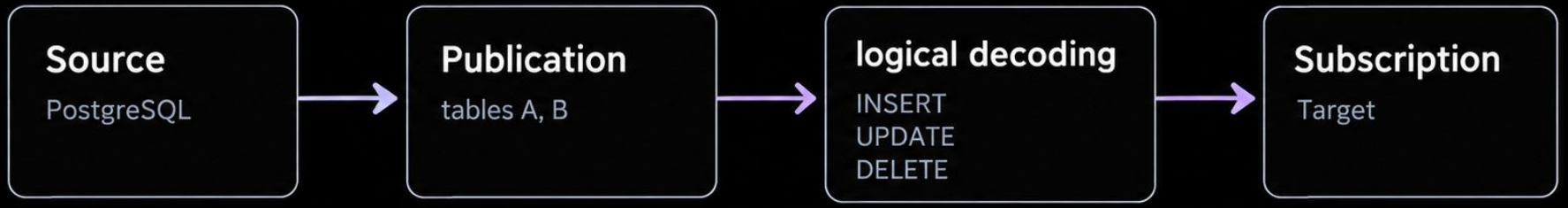


table C поза publication → не реплікується

Physical

- увесь кластер / physical state
- HA · read replicas
- major-version coupling

Logical

- обрані таблиці / row changes
- migration · CDC · upgrade
- schema + sequences окремо

REPLICA ≠ BACKUP

replication чесно копіює і помилки



REPLICA ≠ BACKUP

Replication копіює і правильні зміни, і помилки.

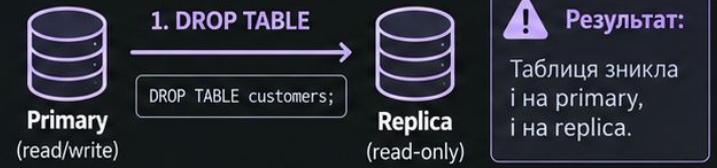
Репліки захищають від відмови вузла, не від помилки людини.

Відновлення: backup → WAL → PITR

Backups дають стартову точку. WAL дозволяє повернутися назад у часі. Разом вони дають point-in-time recovery (PITR).

Replica ≠ backup

Replica — це копія для high availability, а не backup.
Вона сумлінно реплікує зміни — включно з помилками.



Replicas захищають від hardware failure, але не від human error.
Щоб відновитися після помилок, потрібні backups (і WAL).

Шлях відновлення: base backup + WAL → PITR

Віднови новий PostgreSQL instance з base backup, replay-ни WAL і зупинися в target time до поганої транзакції.



Приклад таймлайну point-in-time recovery (PITR)

Відновлення до моменту перед поганою транзакцією.



Результат: ти повертаєш базу в консистентний стан, безпосередньо перед небажаною зміною.

Ключові терміни

RPO (Recovery Point Objective)
Максимально допустима втрата даних.
У контексті PITR RPO зазвичай вимірюється в секундах/хвилинах (залежить від частоти архівування WAL).

RTO (Recovery Time Objective)
Максимально допустимий час відновлення.
Час, необхідний, щоб зробити backup, replay-ни WAL і підняти базу на потрібний момент.

Мета
Використовуй backups + WAL архівування + PITR, щоб відповідати своїм RPO і RTO, і відновлюватися як після інфраструктурних збоїв, так і після помилок людей.

ФІНАЛЬНА МЕНТАЛЬНА МОДЕЛЬ

Сім речей, які не можна плутати.

Partitioning	**≠**	**Sharding**
Replication	**≠**	**Backup**
Replication	**≠**	**Automatic failover**
HA	**≠**	**Disaster recovery**
COMMIT on primary	**≠**	**Immediate replica visibility**
Async replica	**≠**	**Guaranteed RPO = 0**
Base backup + WAL archive	**→**	**PITR**

ПРОБЛЕМА → МЕХАНІЗМ → ТОЧНА ГАРАНТІЯ → ЧОГО НЕ ГАРАНТУЄ → ЦІНА

r_d